ПРАКТИЧНА РОБОТА №1: ОСНОВИ РОБОТИ В PYTHON

Мета роботи – засвоїти базовий синтаксис мови програмування Python.

Теоретичні відомості

Python – це високорівнева динамічна типізована мова програмування із багатьма парадигмами. Код Python дуже схожий із псевдокодом, оскільки він дозволяє висловлювати дуже потужні ідеї в дуже мало рядків коду, при цьому він є дуже читабельним. Як приклад, наведемо реалізацію класичного алгоритму quicksort в Python:

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)
print(quicksort([3,6,8,10,1,2,1]))
# Prints "[1, 1, 2, 3, 6, 8, 10]"
```

Версії Python та необхідні бібліотеки.

Наразі підтримуються та використовуються дві версії Python 2.7 та 3.9. Python 3.0 ввів у мову багато змін, тому код, написаний для версії 2.7, може не працювати під 3.9 і навпаки. Ми будемо використовувати Python версії 3.5 та старше. Ви можете перевірити версію Python, що встановлена на вашому комп'ютері запустивши із командної строки *python --version*. Також для наступної роботи буде доречно встановити програму PyCharm

community (безкоштовна програма) разом із бібліотекою Anaconda (використовуйте офіційні сайти програм для установки, вибравши версію, що відповідає вашій операційній системі та характеристикам комп'ютера *https://www.jetbrains.com/ru-ru/pycharm/download/#section=windows, https://www.anaconda.com/distribution/*). Крім того необхідно установити бібліотеку *pytorch*, це можна зробити із командної строки Windows командою *pip install torch*.

Основні типи даних в Python.

Як і більшість мов, Python має ряд основних типів, включаючи цілі числа, числа з плаваючою комою, булевий тип і рядки. Ці типи даних ведуть себе так, як це знайоме з інших мов програмування.

Числа: Цілі числа чи числа з плаваючою комою працюють так, як ви очікували від інших мов:

```
x = 3
print(type(x)) # Prints "<class 'int'>"
print(x)       # Prints "3"
print(x + 1)   # Addition; prints "4"
print(x - 1)   # Subtraction; prints "2"
print(x * 2)   # Multiplication; prints "6"
print(x ** 2)  # Exponentiation; prints "9"
x += 1
print(x)  # Prints "4"
x *= 2
print(x)  # Prints "8"
y = 2.5
print(type(y)) # Prints "<class 'float'>"
print(y, y + 1, y * 2, y ** 2) # Prints "2.5 3.5 5.0 6.25"
```

Зауважте, що на відміну від багатьох мов, Python не має операторів унарного збільшення (x++) або зменшення (x--).

Python також має вбудовані типи для комплексних чисел.

Booleans: Python реалізує всі звичні для булевої логіки оператори, але використовує англійські слова, а не символи (&&, || тощо):

```
t = True
f = False
print(type(t)) # Prints "<class 'bool'>"
print(t and f) # Logical AND; prints "False"
print(t or f)  # Logical OR; prints "True"
print(not t)   # Logical NOT; prints "False"
print(t != f)  # Logical XOR; prints "True"
Рядки: Python має велику підтримку для рядків:
hello = 'hello'    # String literals can use single quotes
world = "world"    # or double quotes; it does not matter.
print(hello)       # Prints "hello"
print(len(hello))  # String length; prints "5"
hw = hello + ' ' + world  # String concatenation
print(hw)  # prints "hello world"
hw12 = '%s %s %d' % (hello, world, 12)  # sprintf style string formatting
print(hw12)  # prints "hello world 12"
Рядки мають купу корисних методів; наприклад:
s = "hello"
print(s.capitalize())  # Capitalize a string; prints "Hello"
print(s.upper())       # Convert a string to uppercase; prints "HELLO"
print(s.rjust(7))      # Right-justify a string, padding with spaces; prints "
hello"
print(s.center(7))     # Center a string, padding with spaces; prints "hello"
```

print(s.replace('l', '(ell)')) # Replace all instances of one substring with another; prints "he(ell)(ell)o"

print(' world '.strip()) # Strip leading and trailing whitespace; prints "world"

Контейнери Python.

Python включає кілька вбудованих типів контейнерів: списки, словники, набори та кортежі.

1) **Списки**

Список є еквівалентом масиву Python, але його можна змінити за розміром і він може містити елементи різних типів:

```
xs = [3, 1, 2]    # Create a list
print(xs, xs[2])  # Prints "[3, 1, 2] 2"
print(xs[-1])     # Negative indices count from the end of the list; prints "2"
xs[2] = 'foo'     # Lists can contain elements of different types
print(xs)         # Prints "[3, 1, 'foo']"
xs.append('bar')  # Add a new element to the end of the list
print(xs)         # Prints "[3, 1, 'foo', 'bar']"
x = xs.pop()      # Remove and return the last element of the list
print(x, xs)      # Prints "bar [3, 1, 'foo']"
```

Зрізи: крім доступу до елементів списку один за одним, Python надає стислий синтаксис для доступу до списків; це відомо як зрізи:

```
nums = list(range(5))     # range is a built-in function that creates a list of integers
print(nums)               # Prints "[0, 1, 2, 3, 4]"
print(nums[2:4])          # Get a slice from index 2 to 4 (exclusive); prints "[2, 3]"
```

```
print(nums[2:])          # Get a slice from index 2 to the end; prints "[2, 3, 4]"
print(nums[:2])          # Get a slice from the start to index 2 (exclusive); prints "[0, 1]"
print(nums[:])           # Get a slice of the whole list; prints "[0, 1, 2, 3, 4]"
print(nums[:-1])         # Slice indices can be negative; prints "[0, 1, 2, 3]"
nums[2:4] = [8, 9]       # Assign a new sublist to a slice
print(nums)              # Prints "[0, 1, 8, 9, 4]"
```

Ми знову далі побачимо зрізи у контексті нумерованих масивів (numpy arrays).

Цикли: Ви можете переходити до елементів такого списку:

```
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
# Prints "cat", "dog", "monkey", each on its own line.
```

Якщо ви хочете отримати доступ до індексу кожного елемента в тілі циклу, використовуйте вбудовану функцію перерахування (enumerate):

```
animals = ['cat', 'dog', 'monkey']
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
# Prints "#1: cat", "#2: dog", "#3: monkey", each on its own line
```

List comprehensions: часто програмуючи, ми хочемо перетворити один тип даних в інший. Як простий приклад, розглянемо наступний код, який обчислює квадрати чисел:

```
nums = [0, 1, 2, 3, 4]
squares = []
for x in nums:
    squares.append(x ** 2)
print(squares)   # Prints [0, 1, 4, 9, 16]
```

Ви можете зробити цей код простішим використавши list comprehension:

```
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)   # Prints [0, 1, 4, 9, 16]
List comprehensions може також мати умови:
nums = [0, 1, 2, 3, 4]
even_squares = [x ** 2 for x in nums if x % 2 == 0]
print(even_squares)  # Prints "[0, 4, 16]"
```

2) **Словники**

Словник зберігає пари (ключ, значення), схожі з Map на Java або об'єктом у Javascript. Ви можете використовувати його так:

```
d = {'cat': 'cute', 'dog': 'furry'}  # Create a new dictionary with some data
print(d['cat'])       # Get an entry from a dictionary; prints "cute"
print('cat' in d)     # Check if a dictionary has a given key; prints "True"
d['fish'] = 'wet'     # Set an entry in a dictionary
print(d['fish'])      # Prints "wet"
# print(d['monkey'])  # KeyError: 'monkey' not a key of d
print(d.get('monkey', 'N/A'))  # Get an element with a default; prints "N/A"
print(d.get('fish', 'N/A'))    # Get an element with a default; prints "wet"
del d['fish']         # Remove an element from a dictionary
```

```
print(d.get('fish', 'N/A')) # "fish" is no longer a key; prints "N/A"
```

Цикли: легко перебирати ключі в словнику:

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal in d:
    legs = d[animal]
    print('A %s has %d legs' % (animal, legs))
# Prints "A person has 2 legs", "A cat has 4 legs", "A spider has 8 legs"
```

Якщо ви хочете отримати доступ до ключів та відповідних їм значень, використовуйте метод елементів:

```
d = {'person': 2, 'cat': 4, 'spider': 8}
for animal, legs in d.items():
    print('A %s has %d legs' % (animal, legs))
# Prints "A person has 2 legs", "A cat has 4 legs", "A spider has 8 legs"
```

Dictionary comprehensions: вони подібні до List comprehensions, але дозволяють легко будувати словники. Наприклад:

```
nums = [0, 1, 2, 3, 4]
even_num_to_square = {x: x ** 2 for x in nums if x % 2 == 0}
print(even_num_to_square)  # Prints "{0: 0, 2: 4, 4: 16}"
```

3) **Набори**

Набір – це не упорядкована колекція різних елементів. Як простий приклад розглянемо наступне:

```
animals = {'cat', 'dog'}
```

```
print('cat' in animals)   # Check if an element is in a set; prints "True"
print('fish' in animals)  # prints "False"
animals.add('fish')       # Add an element to a set
print('fish' in animals)  # Prints "True"
print(len(animals))       # Number of elements in a set; prints "3"
animals.add('cat')        # Adding an element that is already in the set does nothing
print(len(animals))       # Prints "3"
animals.remove('cat')     # Remove an element from a set
print(len(animals))       # Prints "2"
```

Цикли: Ітерація над набором має той самий синтаксис, що ітерація над списком; однак набори не упорядковані, ви не можете робити припущення про порядок відвідування елементів набору:

```
animals = {'cat', 'dog', 'fish'}
for idx, animal in enumerate(animals):
    print('#%d: %s' % (idx + 1, animal))
# Prints "#1: fish", "#2: dog", "#3: cat"
```

Set comprehensions: подібно до списків та словників, ви можете легко побудувати набор використовуючи set comprehensions:

```
from math import sqrt
nums = {int(sqrt(x)) for x in range(30)}
print(nums)  # Prints "{0, 1, 2, 3, 4, 5}"
```

4) **Кортеж**

Кортеж – це (незмінний) упорядкований список значень. Кортеж багато в чому схожий на список; одна з найважливіших відмінностей

полягає в тому, що кортежі можна використовувати як ключі в словниках і як елементи наборів, тоді як списки не можна. Ось тривіальний приклад:

```
d = {(x, x + 1): x for x in range(10)}  # Create a dictionary with tuple keys
t = (5, 6)        # Create a tuple
print(type(t))    # Prints "<class 'tuple'>"
print(d[t])       # Prints "5"
print(d[(1, 2)])  # Prints "1"
```

Функції Python

Функції Python визначаються за допомогою ключового слова *def*. Наприклад:

```
def sign(x):
    if x > 0:
        return 'positive'
    elif x < 0:
        return 'negative'
    else:
        return 'zero'
for x in [-1, 0, 1]:
    print(sign(x))
# Prints "negative", "zero", "positive"
```

Ми часто визначаємо функції, які можуть приймати необов'язкові (keyword) аргументи:

```
def hello(name, loud=False):
    if loud:
        print('HELLO, %s!' % name.upper())
```

```
    else:
        print('Hello, %s' % name)
hello('Bob') # Prints "Hello, Bob"
hello('Fred', loud=True)  # Prints "HELLO, FRED!"
```

Класи Python

Синтаксис для визначення класів у Python є простим:

```
class Greeter(object):
    # Constructor
    def __init__(self, name):
        self.name = name  # Create an instance variable
    # Instance method
    def greet(self, loud=False):
        if loud:
            print('HELLO, %s!' % self.name.upper())
        else:
            print('Hello, %s' % self.name)
g = Greeter('Fred')  # Construct an instance of the Greeter class
g.greet()           # Call an instance method; prints "Hello, Fred"
g.greet(loud=True)   # Call an instance method; prints "HELLO, FRED!"
```

Практична частина

1. Написати програму на Python, яка:
   a) Сортує словник за значеннями
   b) Додає ключ до словника
      Було : {0: 10, 1: 20}
      Результат : {0: 10, 1: 20, 2: 30}
   c) Поєднає наступні словники та створює новий
      dic1={1:10, 2:20}

dic2={3:30, 4:40}

dic3={5:50,6:60}

Результат : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

d) Перевіряє чи входить даний ключ до словника
e) Ітерується по словнику використовуючи for loops.
f) Генерує та виводить словник, який складається с чисел від 1 до n в формі (x, x*x).

   Ввід (n = 5). Результат : {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
g) Генерує словник у якому ключі це числа від 1 до 15, а значення квадрати ключів.

   Приклад {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100, 11: 121, 12: 144, 13: 169, 14: 196, 15: 225}
h) Яка створює будь який set.
i) Яка ітерується крізь sets.
j) Додає значення до set.
k) Видаляє елемент з set
l) Видаляє елемент якщо він є в set.
m) Створює об’єднання sets (intersection).
n) Яка знаходить суму всіх елементів списку.
o) Яка помножує всі елементи списку
p) Яка виводить найбільше та найменше значення у списку.
q) яка отримує список, відсортований у порядку зростання за останнім елементом у кожному кортежі із заданого списку не порожніх кортежів.

   Приклад : [(2, 5), (1, 2), (4, 4), (2, 3), (2, 1)]

   Результат : [(2, 1), (1, 2), (2, 3), (4, 4), (2, 5)]
r) Яка видаляє елементи які повторюються
s) Яка друкує лист після видалення 0го, 4го та 5го елементів.

   Приклад : ['Red', 'Green', 'White', 'Black', 'Pink', 'Yellow']

   Результат : ['Green', 'White', 'Black']

t) Яка помножує всі значення в списку.

Приклад: (8, 2, 3, -1, 7). Результат: -336

u) Розраховує факторіал числа.

v) Приймає строку та підраховує кількість великих та маленьких букв.

Ввід : 'The quick Brow Fox'

Результат : No. of Upper case characters : 3 No. of Lower case Characters : 12

w) Друкує наступний патерн, використовуючи цикли

```
*
 *
 * *
 * * *
 * * * *
 * * *
 * *
 *
*
```

2. Розписати коротко під кожен пункт, що може бути використано для заданого варіанту.

Висновки

У висновках обґрунтувати переваги та недоліки мови програмування Python.

Контрольні запитання

1. Що таке словник?
2. Що таке sets?
3. Що таке кортеж?